

Banco de Dados Relacionais

Guia Completo com MySQL Workbench

Uma apostila completa para dominar bancos de dados relacionais seguindo as normas ABNT

Índice

01

Fundamentos de Bancos de Dados

Introdução, conceitos e estruturas básicas

02

Modelagem e Design SQL Essencial

Entidades, relacionamentos e normalização

03

Comandos e cláusulas fundamentais

04

Consultas Avançadas

JOINS, views e procedimentos

05

armazenados

MySQL Workbench Ferramentas

práticas e RAID

Este material foi desenvolvido especialmente para estudantes que precisam compreender bancos de dados relacionais de forma prática e aplicada, seguindo as melhores práticas da indústria e os padrões acadêmicos brasileiros.

Introdução a Bancos de Dados Relacionais

Um banco de dados relacional é um sistema organizado de armazenamento de informações que utiliza tabelas para estruturar dados de maneira lógica e eficiente. Desenvolvido originalmente por Edgar F. Codd na década de 1970, este modelo revolucionou a forma como as organizações gerenciam informações, tornando-se o padrão dominante na indústria de tecnologia.

O conceito fundamental por trás dos bancos de dados relacionais é a organização de dados em tabelas bidimensionais, onde cada linha representa um registro único e cada coluna representa um atributo específico desse registro. Esta estrutura tabular permite que os dados sejam facilmente compreendidos, manipulados e relacionados entre si através de chaves que estabelecem conexões lógicas.

A força do modelo relacional reside em sua capacidade de reduzir redundância, manter a integridade dos dados e facilitar consultas complexas através de uma linguagem padronizada chamada SQL (Structured Query Language). Empresas de todos os tamanhos, desde startups até corporações multinacionais, dependem de bancos de dados relacionais para gerenciar desde informações de clientes até transações financeiras críticas.

Segundo as normas ABNT NBR ISO/IEC 2382, um banco de dados é definido como um conjunto de dados inter-relacionados, armazenados sem redundância desnecessária, para atender a múltiplas aplicações. O MySQL, um dos sistemas de gerenciamento de banco de dados relacionais mais populares do mundo, oferece uma implementação robusta e acessível deste modelo, sendo amplamente utilizado tanto em ambientes acadêmicos quanto corporativos.

Características Fundamentais



Estrutura Tabular

Organização de dados em linhas e colunas que facilita visualização e manipulação

Regras que garantem consistência e precisão das informações armazenadas

Relacionamentos

Conexões entre tabelas através de chaves que mantêm a integridade referencial

Integridade de Dados

Padrão universal para consulta e
manipulação de dados
relacionais

Linguagem SQL

Estas características trabalham em conjunto para criar um sistema robusto e confiável de gerenciamento de informações, permitindo que desenvolvedores e analistas construam aplicações escaláveis e eficientes.

Vantagens do Modelo Relacional

existentes.

Flexibilidade

O modelo relacional permite modificar a estrutura do banco de dados sem afetar as aplicações existentes. Novas tabelas podem ser adicionadas e relacionamentos podem ser estabelecidos conforme necessário.

Esta flexibilidade é essencial em ambientes onde os requisitos de negócio evoluem constantemente, permitindo adaptações rápidas sem comprometer a integridade dos dados

Redução de Redundância

Através da normalização, o modelo relacional minimiza a duplicação de dados, economizando espaço de armazenamento e reduzindo inconsistências.

Cada informação é armazenada em um único local e referenciada onde necessário, garantindo que atualizações sejam refletidas automaticamente em todo o sistema.

O Que São Anomalias em Bancos de Dados

Anomalias são problemas que ocorrem em bancos de dados mal estruturados, resultando em inconsistências, redundâncias e dificuldades operacionais. Estas situações indesejadas surgem

quando a modelagem de dados não segue princípios adequados de normalização, levando a complicações durante operações de inserção, atualização ou exclusão de registros.

Compreender anomalias é fundamental para desenvolver sistemas de banco de dados robustos e eficientes. Quando não tratadas adequadamente, estas irregularidades podem causar perda de informação, desperdício de recursos de armazenamento e inconsistências que comprometem a confiabilidade dos dados. A identificação e correção de anomalias através de técnicas de normalização é uma etapa essencial no processo de design de bancos de dados.

As anomalias são classificadas em três categorias principais: anomalias de inserção, que ocorrem quando não é possível adicionar dados sem informações adicionais desnecessárias; anomalias de atualização, que surgem quando a modificação de um dado requer alterações em múltiplos locais; e anomalias de exclusão, que acontecem quando a remoção de um registro causa perda não intencional de outras informações importantes.

O estudo das anomalias está diretamente relacionado ao processo de normalização de bancos de dados, que será abordado em seções posteriores desta apostila. Reconhecer estas situações problemáticas é o primeiro passo para construir estruturas de dados mais eficientes e confiáveis.

Tipos de Anomalias

	cadastrar um professor sem alunos	várias linhas
Anomalia de Inserção	Anomalia de Atualização	Anomalia de Exclusão
Impossibilidade de adicionar dados sem incluir informações irrelevantes ou incompletas	Necessidade de modificar múltiplos registros para atualizar uma única informação	Perda não intencional de informações ao remover um registro
Requer dados adicionais não relacionados	Gera redundância de operações	Elimina dados relacionados indesejavelmente
Cria dependências desnecessárias	Aumenta risco de inconsistências	Compromete integridade histórica
Exemplo: não poder	Exemplo: alterar endereço repetido em	Exemplo: excluir último aluno e perder dados do curso

Exemplo Prático de Anomalias

Considere uma tabela mal estruturada que armazena informações de alunos e cursos juntos: **ID_Aluno Nome_Aluno Curso Dept_Curso Professor**

1 João Silva Banco de Dados

2 Ana Costa Banco de Dados

3 Pedro Santos Programação Computação Prof. João
informações do curso de
Programação

Problemas Identificados

Redundância: Informações do curso repetidas para cada aluno

Inserção: Não é possível cadastrar um curso sem alunos matriculados

Atualização: Mudar o professor requer alterar múltiplas linhas

Exclusão: Remover Pedro apaga

Solução Proposta

Separar em três tabelas normalizadas: Alunos, Cursos e Matrículas. Cada entidade terá seus próprios atributos, e relacionamentos serão estabelecidos através de chaves estrangeiras.

Esta abordagem elimina redundância e permite operações independentes sem perda de dados.

Entidades em Bancos de Dados

Uma entidade é um objeto ou conceito do mundo real que possui existência independente e sobre o qual desejamos armazenar informações em nosso banco de dados. Entidades representam os substantivos do nosso modelo de dados - são as "coisas" sobre as quais coletamos e mantemos dados. Por exemplo, em um sistema universitário, entidades típicas incluem Aluno, Professor, Curso e Departamento.

Cada entidade é caracterizada por um conjunto de atributos que descrevem suas propriedades. Um Aluno, por exemplo, pode ter atributos como matrícula, nome, data de nascimento, endereço e telefone. No modelo relacional, cada entidade é geralmente implementada como uma tabela, onde cada linha representa uma instância específica dessa entidade (um aluno particular) e cada coluna representa um atributo.

A identificação correta das entidades é um dos passos mais importantes no processo de modelagem de dados. Entidades devem representar conceitos significativos para o negócio e devem ter um identificador único que permita distinguir uma instância de outra. Este identificador único é conhecido como chave primária e garante que cada registro na tabela seja inequivocamente identificável.

Segundo a notação de Peter Chen, amplamente utilizada em diagramas entidade-relacionamento (DER), entidades são representadas por retângulos. Esta representação visual facilita a comunicação entre desenvolvedores, analistas e stakeholders durante o processo de design do banco de dados, tornando mais clara a estrutura e as relações entre os diferentes elementos do sistema.

Características das Entidades

Identificação Única

Cada entidade deve possuir um identificador que a distingue de todas as outras instâncias. Este atributo

identificador é essencial para garantir a integridade e permitir referências precisas.

armazenam. Os atributos devem ser relevantes e suficientes para representar completamente a entidade no contexto do sistema.

Existência Independente

Uma entidade forte existe por si mesma, não dependendo de outras entidades para sua identificação ou existência no banco de dados.

Conjunto de Atributos

Entidades são definidas pelos dados que

Múltiplas Instâncias

Uma entidade representa um tipo ou classe de objetos, podendo ter várias ocorrências concretas armazenadas como registros individuais na tabela.

Exemplos de Entidades

	Email	institucional
ALUNO	PROFESSOR	CURSO
Atributos:	Atributos:	Atributos:
Matrícula	Código	Código do curso
(identificador) Nome completo	(identificador) Nome completo	(identificador)
Data de nascimento	Titulação	Nome do curso
CPF	Departamento	Carga horária
Endereço	Salário	Departamento
Telefone	Data de contratação	Nível
	Email	(graduação/pós)
		Modalidade

Entidades Fracas

Uma entidade fraca é uma entidade especial que não possui um identificador próprio suficiente para distinguir suas instâncias de forma única. Diferentemente das entidades fortes, que existem de maneira independente, as entidades fracas dependem da existência de outra entidade, chamada entidade proprietária ou identificadora, para sua completa identificação.

A característica fundamental de uma entidade fraca é sua dependência existencial: ela não pode existir no banco de dados sem estar associada a uma instância da entidade forte da qual depende. Esta dependência é representada através de um relacionamento identificador, que conecta a entidade fraca à sua entidade proprietária. Por exemplo, em um sistema bancário, a entidade DEPENDENTE é fraca em relação à entidade FUNCIONÁRIO, pois um dependente só existe no contexto de um funcionário específico.

Para identificar completamente uma instância de entidade fraca, é necessário combinar seu identificador parcial (também chamado de discriminador) com a chave primária da entidade proprietária. Este identificador parcial sozinho não é único dentro de toda a tabela, mas é único

dentro do contexto da entidade proprietária. Por exemplo, vários funcionários podem ter um dependente chamado "filho 1", mas a combinação de ID do funcionário + "filho 1" é única.

Em diagramas entidade-relacionamento, entidades fracas são tradicionalmente representadas por retângulos duplos, e o relacionamento identificador é indicado por um losango duplo. Esta notação visual ajuda a distinguir rapidamente quais entidades possuem dependência existencial e quais são independentes. Compreender entidades fracas é essencial para modelar corretamente situações onde a existência de um dado está intrinsecamente ligada à existência de outro.

Características das Entidades

Fracas Dependência Existencial



Não pode existir sem a entidade proprietária - sua existência depende completamente de outra entidade

Identificador Parcial



Possui apenas um discriminador que, sozinho, não identifica unicamente a entidade

Relacionamento Identificador

Conectada à entidade forte através de um relacionamento obrigatório de identificação

Chave Composta

Identificação completa através da combinação do discriminador com a chave da entidade forte

Exemplo Prático: Entidade Fraca

Cenário

Em um sistema de RH, queremos registrar os dependentes de cada funcionário.

Entidade Forte: FUNCIONÁRIO

ID_Funcionário (PK)

Nome

CPF

Cargo

Entidade Fraca: DEPENDENTE

Nome_Dependente
(discriminador)

Data_Nascimento

Parentesco

ID_Funcionário (FK + PK)

Por que é Entidade Fraca?

Um dependente não existe sem um funcionário associado. O nome do dependente sozinho não é único - dois funcionários diferentes podem ter dependentes com o mesmo nome.

A identificação completa requer: **ID_Funcionário**
+ **Nome_Dependente**

Se um funcionário for removido do sistema, todos os seus dependentes também devem ser removidos, pois não têm sentido existir

independentemente.

Implementação SQL

```
CREATE TABLE Dependente (  
  ID_Funcionario INT,  
  Nome_Dependente VARCHAR(100),  
  Data_Nascimento DATE,  
  Parentesco VARCHAR(50),  
  PRIMARY KEY (ID_Funcionario,  
  Nome_Dependente), FOREIGN KEY  
  (ID_Funcionario)  
  REFERENCES Funcionario(ID_Funcionario)  
  ON DELETE CASCADE  
);
```

Chave Primária (Primary Key)

A chave primária é um atributo ou conjunto de atributos que identifica de forma única cada registro em uma tabela. É um dos conceitos mais fundamentais em bancos de dados relacionais, garantindo que cada linha da tabela possa ser distinguida inequivocamente de todas as outras. A chave primária estabelece a identidade de cada entidade e serve como referência para relacionamentos com outras tabelas.

Uma chave primária deve satisfazer três propriedades essenciais: unicidade, onde nenhum valor pode se repetir entre os registros; não-nulidade, significando que não pode conter valores nulos ou vazios; e minimalidade, indicando que deve usar o menor conjunto possível de atributos necessários para garantir a unicidade. Estas características asseguram a integridade e confiabilidade dos dados armazenados.

Existem dois tipos principais de chaves primárias: simples e compostas. Uma chave primária simples consiste em apenas um atributo, como um número de matrícula ou CPF. Já uma chave primária composta é formada pela combinação de dois ou mais atributos, usada quando nenhum atributo individual é suficiente para garantir unicidade. Por exemplo, em uma tabela de matrículas, a combinação de ID_Aluno e Código_Disciplina pode formar a chave primária.

A escolha adequada da chave primária impacta diretamente o desempenho e a integridade do banco de dados. É preferível usar chaves que sejam imutáveis (não mudam ao longo do tempo) e sem significado semântico (números sequenciais ou códigos gerados automaticamente), conhecidas como chaves substitutas, em vez de chaves naturais que possuem significado no mundo real e podem estar sujeitas a mudanças.

Propriedades da Chave Primária

Unicidade

Cada valor da chave primária deve ser único na tabela. Não podem existir duas

linhas com o mesmo valor de chave primária. Esta propriedade garante identificação inequívoca de cada registro.

Imutabilidade

Idealmente, o valor da chave primária não deve mudar ao longo do tempo. Alterações na chave primária podem causar problemas de integridade referencial e complicar manutenção do banco.

Não-Nulidade

A chave primária nunca pode conter valores NULL. Todo registro deve ter um valor válido e definido para sua chave primária, garantindo que cada entidade seja sempre identificável.

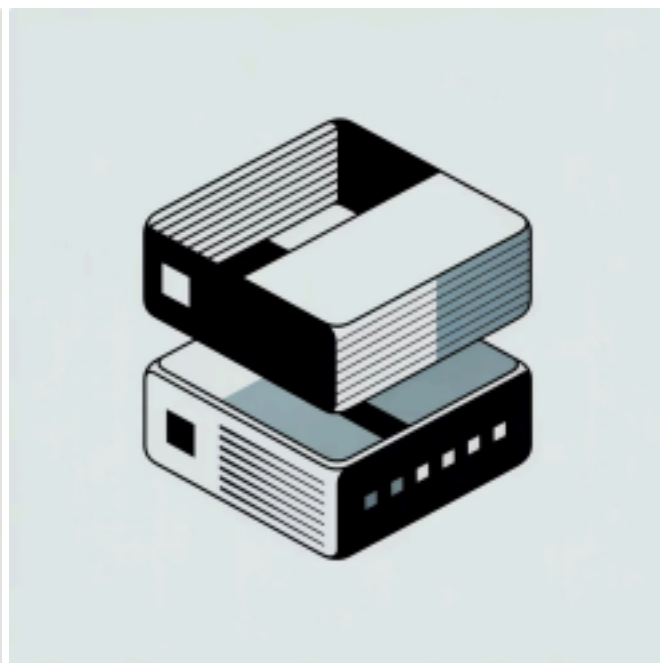
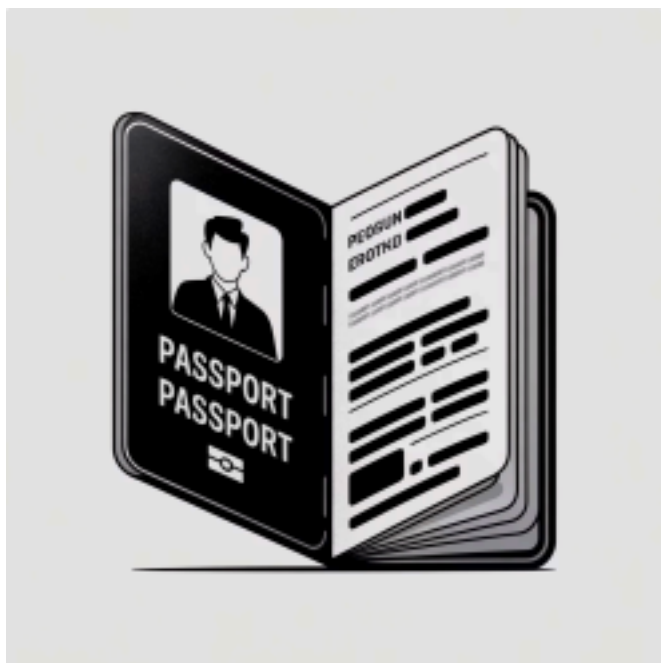
Minimalidade

Deve usar o menor conjunto possível de atributos. Se um único atributo é suficiente para garantir unicidade, não deve-se usar múltiplos atributos na chave primária.

Tipos de Chaves Primárias

Chave Substituta

Chave Natural



Atributo que possui significado no mundo real e pode ser usado como identificador.

Exemplos:

- CPF em tabela de Pessoas
- Placa em tabela de Veículos
- ISBN em tabela de Livros
- Matrícula em tabela de Alunos

Vantagens: Significado intuitivo, facilita compreensão

Desvantagens: Pode mudar, pode ser

sensível, requer validação externa

Identificador artificial criado especificamente para servir como chave primária, sem significado no mundo real.

Exemplos:

- ID auto-incrementado
- UUID (identificador universal único)
- Código sequencial gerado
- Hash único gerado

Vantagens: Imutável, sem restrições

externas, performance otimizada

requer índice adicional para buscas
semânticas

Desvantagens: Não tem significado direto,

Implementação de Chaves Primárias

Exemplos em SQL

-- Chave primária simples com auto-incremento

```
CREATE TABLE Aluno (  
  ID_Aluno INT AUTO_INCREMENT PRIMARY KEY,  
  Nome VARCHAR(100) NOT NULL,  
  CPF VARCHAR(11) UNIQUE,  
  Data_Nascimento DATE  
);
```

-- Chave primária composta

```
CREATE TABLE Matricula (  
  ID_Aluno INT,  
  Codigo_Disciplina VARCHAR(10),  
  Semestre VARCHAR(6),  
  Nota DECIMAL(4,2),  
  PRIMARY KEY (ID_Aluno, Codigo_Disciplina, Semestre)  
);
```

-- Definição de chave primária após criação da tabela

```
CREATE TABLE Professor (  
  Codigo_Professor INT,  
  Nome VARCHAR(100),  
  Departamento VARCHAR(50)  
);
```

```
ALTER TABLE Professor  
ADD PRIMARY KEY (Codigo_Professor);
```

Note que em MySQL, o uso de AUTO_INCREMENT automatiza a geração de valores únicos sequenciais, simplificando a criação de chaves substitutas.

Chave Estrangeira (Foreign Key)

A chave estrangeira é um atributo ou conjunto de atributos em uma tabela que referencia a chave primária de outra tabela, estabelecendo um relacionamento entre as duas. É o mecanismo fundamental que implementa relacionamentos em bancos de dados relacionais, permitindo que dados sejam conectados logicamente através de diferentes tabelas enquanto mantêm a integridade referencial do sistema.

A principal função da chave estrangeira é garantir a integridade referencial, um princípio que assegura que relacionamentos entre tabelas permaneçam consistentes. Quando uma chave estrangeira é definida, o sistema de gerenciamento de banco de dados automaticamente verifica que os valores inseridos ou atualizados na coluna de chave estrangeira existem como chave primária na tabela referenciada. Isso previne a criação de "órfãos" - registros que referenciam entidades inexistentes.

Por exemplo, em um sistema acadêmico, a tabela MATRICULA pode ter uma chave estrangeira ID_Aluno que referencia a chave primária da tabela ALUNO. Isso garante que não seja possível matricular um aluno que não existe no banco de dados. Além disso, a chave estrangeira pode ser configurada com regras de ação em cascata, determinando o que acontece quando um registro referenciado é atualizado ou excluído.

As ações em cascata incluem: CASCADE (propaga a operação para os registros dependentes), SET NULL (define a chave estrangeira como NULL), SET DEFAULT (define um valor padrão), RESTRICT (impede a operação se houver registros dependentes), e NO ACTION (similar a RESTRICT). A escolha apropriada dessas ações é crucial para manter a integridade dos dados conforme as regras de negócio do sistema.

Características da Chave

Estrangeira Referência Obrigatória

1

Deve apontar para uma chave primária ou candidata existente em outra tabela (ou na mesma tabela em auto-relacionamentos)

Integridade Referencial

2

Garante que valores inseridos existem na tabela referenciada, prevenindo inconsistências de dados

Pode Ser Nula

3

Diferentemente da chave primária, pode aceitar valores NULL quando o relacionamento é opcional

Permite Duplicação

4

Múltiplos registros podem ter o mesmo valor de chave estrangeira (relação um-para

muitos)

Ações em Cascata

5

Define comportamento automático quando registro referenciado é modificado ou excluído

Ações em Cascata da Chave Estrangeira

Ação Comportamento Exemplo de Uso

CASCADE	Propaga alteração ou exclusão para registros dependentes	integridade após a transação Excluir um cliente exclui automaticamente seus pedidos
SET NULL	Define chave estrangeira como NULL quando registro pai é removido	Remover departamento deixa funcionários sem departamento
SET DEFAULT	Atribui valor padrão à chave estrangeira	Define departamento padrão ao remover departamento original
RESTRICT	Impede exclusão/atualização se houver dependentes	Não permite excluir categoria com produtos associados
NO ACTION	Similar a RESTRICT, verifica	Validação posterior para operações complexas

A escolha da ação apropriada depende das regras de negócio. CASCADE é útil para dependências fortes, enquanto RESTRICT protege contra exclusões acidentais de dados importantes.

Implementação de Chaves Estrangeiras

```
-- Criação com chave estrangeira inline
CREATE TABLE Pedido (
  ID_Pedido INT PRIMARY KEY AUTO_INCREMENT,
  Data_Pedido DATE NOT NULL,
  ID_Cliente INT NOT NULL,
  Valor_Total DECIMAL(10,2),
  FOREIGN KEY (ID_Cliente)
  REFERENCES Cliente(ID_Cliente)
  ON DELETE RESTRICT
```

```
ON UPDATE CASCADE
```

```
);
```

```
-- Adicionando chave estrangeira após criação
```

```
CREATE TABLE Item_Pedido (
```

```
ID_Item INT PRIMARY KEY AUTO_INCREMENT,
```

```
ID_Pedido INT,
```

```
ID_Produto INT,
```

```
Quantidade INT,
```

```
Preco_Unitario DECIMAL(10,2)
```

```
);
```

```
ALTER TABLE Item_Pedido
```

```
ADD CONSTRAINT FK_Pedido
```

```
FOREIGN KEY (ID_Pedido)
```

```
REFERENCES Pedido(ID_Pedido)
```

```
ON DELETE CASCADE;
```

```
ALTER TABLE Item_Pedido
```

```
ADD CONSTRAINT FK_Produto
```

```
FOREIGN KEY (ID_Produto)
```

```
REFERENCES Produto(ID_Produto)
```

```
ON DELETE RESTRICT;
```

Relacionamentos em Bancos de Dados

Relacionamentos representam as associações lógicas entre entidades em um banco de dados, descrevendo como os dados de diferentes tabelas se conectam e interagem. São a essência do modelo relacional, permitindo que informações distribuídas em múltiplas tabelas sejam logicamente relacionadas e recuperadas de forma integrada através de consultas. Os relacionamentos refletem as regras de negócio e a natureza das interações entre as entidades do sistema.

Um relacionamento é implementado através de chaves estrangeiras que conectam a chave primária de uma tabela a atributos correspondentes em outra tabela. Por exemplo, o relacionamento entre ALUNO e CURSO em um sistema acadêmico indica quais alunos estão matriculados em quais cursos. Esta conexão não apenas estabelece a associação, mas também permite navegar entre os dados relacionados, recuperando informações completas que abrangem múltiplas entidades.

Os relacionamentos são classificados em três tipos principais baseados em cardinalidade: um para-um (1:1), onde cada instância de uma entidade relaciona-se com no máximo uma instância de outra; um-para-muitos (1:N), onde uma instância pode relacionar-se com múltiplas instâncias da outra entidade; e muitos-para-muitos (N:M), onde múltiplas instâncias de ambas as entidades

podem estar relacionadas entre si.

Compreender e modelar corretamente os relacionamentos é fundamental para criar bancos de dados eficientes e que representem fielmente a realidade do negócio. Relacionamentos mal projetados podem levar a redundância, anomalias e dificuldades nas consultas, enquanto relacionamentos bem estruturados facilitam a manutenção, garantem integridade dos dados e otimizam o desempenho das operações.

Tipos de Relacionamentos

	pertence a apenas uma pessoa.	cliente.
Um-para-Um (1:1)	Um-para-Muitos (1:N)	Muitos-para-Muitos (N:M)
Cada instância de uma entidade relaciona-se com no máximo uma instância da outra entidade.	Uma instância de uma entidade pode relacionar-se com múltiplas instâncias da outra, mas não vice-versa.	Múltiplas instâncias de ambas as entidades podem relacionar-se entre si.
Exemplo: Uma pessoa possui um único CPF, e cada CPF	Exemplo: Um cliente pode fazer vários pedidos, mas cada pedido pertence a apenas um	Exemplo: Alunos podem matricular-se em vários cursos, e cada curso tem vários alunos.

Implementação de Relacionamentos

Relacionamento 1:N

Implementado colocando a chave estrangeira na tabela do lado "muitos".

```
CREATE TABLE Departamento (  
  ID_Depto INT PRIMARY KEY,  
  Nome VARCHAR(100)  
);  
  
CREATE TABLE Funcionario (  
  ID_Func INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  ID_Depto INT,  
  FOREIGN KEY (ID_Depto)  
  REFERENCES Departamento(ID_Depto) );
```

Um departamento tem muitos funcionários, mas cada funcionário pertence a um departamento.

Relacionamento N:M

Requer tabela intermediária (tabela associativa) com chaves estrangeiras para ambas as entidades.

```
CREATE TABLE Aluno (  
  ID_Aluno INT PRIMARY KEY,  
  Nome VARCHAR(100)  
);  
  
CREATE TABLE Disciplina (  
  Cod_Disciplina VARCHAR(10) PRIMARY  
  KEY, Nome VARCHAR(100)  
);  
  
CREATE TABLE Matricula (  
  ID_Aluno INT,  
  Cod_Disciplina VARCHAR(10),  
  Semestre VARCHAR(6),  
  PRIMARY KEY (ID_Aluno,  
  Cod_Disciplina), FOREIGN KEY
```

(ID_Aluno)
REFERENCES Aluno(ID_Aluno),
FOREIGN KEY (Cod_Disciplina)

REFERENCES
Disciplina(Cod_Disciplina));

Cardinalidade em Relacionamentos

Cardinalidade define o número de instâncias de uma entidade que podem estar associadas a instâncias de outra entidade em um relacionamento. É expressa através de dois componentes: a cardinalidade mínima (também chamada de participação) e a cardinalidade máxima, que juntas especificam as restrições quantitativas do relacionamento. Compreender e especificar corretamente a cardinalidade é essencial para garantir que o banco de dados reflita precisamente as regras de negócio.

A cardinalidade mínima indica se a participação no relacionamento é obrigatória ou opcional. Uma cardinalidade mínima de 0 significa participação opcional - uma instância da entidade pode existir sem estar relacionada. Uma cardinalidade mínima de 1 indica participação obrigatória - toda instância deve estar relacionada. Por exemplo, em um sistema universitário, se todo aluno deve estar matriculado em pelo menos uma disciplina, a cardinalidade mínima do aluno no relacionamento de matrícula é 1.

A cardinalidade máxima especifica o número máximo de associações permitidas, tipicamente representada como 1 (no máximo um) ou N/*(muitos, sem limite específico). Combinando cardinalidades mínima e máxima, obtemos notações como (0,1), (1,1), (0,N) e (1,N), onde o primeiro número representa o mínimo e o segundo o máximo. Por exemplo, a notação (1,N) indica que cada instância deve estar relacionada com pelo menos uma e pode estar relacionada com várias instâncias da outra entidade.

A cardinalidade impacta diretamente a implementação física do banco de dados, determinando onde as chaves estrangeiras são colocadas e se valores NULL são permitidos. Também afeta as regras de integridade referencial, definindo se exclusões devem ser restritas ou propagadas em cascata. Especificar cardinalidades incorretamente pode resultar em um banco de dados que não suporta adequadamente os processos de negócio ou que permite estados de dados inválidos.

Notações de Cardinalidade

(0,1)

Zero ou Um

Participação
opcional, no
máximo uma
associação

Exemplo:
Funcionário pode
ter ou não um carro
da empresa

1

(1,1)
(0,N)

Zero ou Muitos

Participação
opcional, múltiplas
associações
possíveis

Exemplo: Cliente
pode ter zero ou
vários pedidos

3

4

2

(1,N)

Exatamente Um

Participação obrigatória, apenas uma associação

Exemplo: Cada pedido deve ter exatamente um cliente

Um ou Muitos

Participação obrigatória, múltiplas associações requeridas

Exemplo: Departamento deve ter pelo menos um funcionário

Exemplos Práticos de Cardinalidade

Entidade A Card. A Entidade B Card. B

Significado

Relacionamento

Casamento Pessoa (0,1) Pessoa (0,1) Uma pessoa

pode ser casada ou não com outra pessoa

Faz Pedido Cliente (1,N) Pedido (1,1) Cliente faz um ou mais

pedidos; pedido tem um cliente

Leciona Professor (1,N) Disciplina (0,N) Professor leciona pelo

menos uma disciplina; disciplina pode não ter professor

Matrícula Aluno (1,N) Curso (0,N) Aluno matricula

se em um ou mais cursos; curso pode não ter alunos

Gerencia Funcionário (0,1) Departamento (1,1) Funcionário pode gerenciar

um departamento; todo departamento

Normalização de Bancos de Dados

Normalização é o processo sistemático de organizar dados em um banco de dados para reduzir redundância e melhorar a integridade. Desenvolvida por Edgar F. Codd, a normalização aplica uma série de regras chamadas formas normais para estruturar tabelas de maneira eficiente. O objetivo principal é eliminar anomalias de inserção, atualização e exclusão, garantindo que cada informação seja armazenada em apenas um local lógico do banco de dados.

O processo de normalização divide tabelas grandes e complexas em tabelas menores e mais especializadas, conectadas através de relacionamentos. Cada forma normal representa um nível crescente de organização e restrições estruturais. Uma tabela normalizada até a terceira forma normal (3FN), que é o padrão mais comum na prática, elimina a maioria dos problemas de redundância e inconsistência encontrados em bancos de dados não normalizados.

A normalização oferece diversos benefícios: redução do espaço de armazenamento ao eliminar dados duplicados; facilidade de manutenção, pois alterações precisam ser feitas em apenas um local; consistência de dados, evitando informações conflitantes; e flexibilidade para expansão futura do banco de dados. Além disso, tabelas normalizadas geralmente apresentam melhor desempenho em operações de inserção, atualização e exclusão.

Embora a normalização seja fundamental para um design robusto de banco de dados, em alguns casos específicos pode-se optar por uma desnormalização controlada para otimizar o desempenho de leitura em sistemas de alta demanda. No entanto, esta decisão deve ser tomada conscientemente, entendendo os trade-offs envolvidos. Para a maioria dos sistemas, especialmente acadêmicos e transacionais, normalizar até a 3FN é considerado boa prática.

Formas Normais - Visão Geral

Formas Normais		
Primeira Forma Normal (1FN)	Segunda Forma Normal (2FN)	Terceira Forma Normal (3FN)
Elimina grupos repetitivos e garante atomicidade dos valores	Elimina dependências parciais da chave primária	Elimina dependências transitivas entre atributos
Cada coluna contém valores atômicos	Cada linha é única	Relevante para chaves compostas
Não há valores múltiplos em uma	Deve estar em 1FN	Deve estar em 2FN
	Todos atributos não	

Nenhum atributo não
chave depende de outro

atributo não-chave
Padrão recomendado

para maioria dos
bancos

Existem formas normais adicionais (FNBC, 4FN, 5FN), mas na prática a 3FN atende a maioria das necessidades de negócio.

Primeira Forma Normal (1FN)

A Primeira Forma Normal estabelece os requisitos básicos para que uma tabela seja considerada relacional. Para estar em 1FN, uma tabela deve satisfazer três critérios fundamentais: todos os atributos devem conter apenas valores atômicos (indivisíveis), não pode haver grupos repetitivos de colunas, e deve existir uma chave primária que identifique unicamente cada registro. Esta forma normal elimina a complexidade de dados multivalorados e estruturas aninhadas.

Violação da 1FN - Exemplo

ID_Aluno	Nome	Telefones	Cursos
1	João Silva	1111-1111, 2222-2222	Matemática, Física
2	Maria Santos	3333-3333	Química, Biologia, História

Problemas: Múltiplos valores em uma célula (Telefones e Cursos não são atômicos), dificuldade para buscar um curso específico, impossível adicionar restrições adequadas.

Tabela Normalizada em 1FN

Tabela: Telefone_Aluno

Tabela: Aluno

ID_Aluno	Telefone
1	1111-1111
1	2222-2222
2	3333-3333

ID_Aluno	Nome
1	João Silva
2	Maria Santos

Segunda Forma Normal (2FN)

A Segunda Forma Normal elimina dependências parciais, que ocorrem quando um atributo não chave depende apenas de parte de uma chave primária composta. Para estar em 2FN, a tabela deve primeiro estar em 1FN e todos os atributos não-chave devem depender funcionalmente da chave primária completa, não apenas de parte dela. Esta forma normal é relevante especialmente para tabelas com chaves primárias compostas por múltiplos atributos.

Violação da 2FN - Exemplo

		Nome_Aluno	Nome_Disciplina	Nota
ID_Aluno	Cod_Disciplina			
1	BD101	João Silva	Banco de Dados	8.5
	PROG101	João Silva	Programação 9.0	
2	BD101	Maria Santos	Banco de Dados	7.5
	PROG101	Maria Santos	Programação 9.0	

Problema: Chave primária é (ID_Aluno, Cod_Disciplina), mas Nome_Aluno depende apenas de ID_Aluno, e Nome_Disciplina depende apenas de Cod_Disciplina. Isso causa redundância e anomalias.

Tabelas Normalizadas em 2FN

		Disciplina	Aluno	
ID_Aluno	Nome_Aluno	Cod_Disciplina	Nome_Disciplina	Matrícula
1	João Silva	BD101	Banco de Dados	8.5
		PROG101	Programação 9.0	7.5
2	Maria Santos	BD101	Banco de Dados	7.5
		PROG101	Programação 9.0	8.5

Terceira Forma Normal (3FN)

A Terceira Forma Normal elimina dependências transitivas, que ocorrem quando um atributo não chave depende de outro atributo não-chave, em vez de depender diretamente da chave primária. Para estar em 3FN, a tabela deve estar em 2FN e nenhum atributo não-chave pode depender de outro atributo não-chave. Esta é considerada a forma normal padrão para a maioria dos bancos de dados transacionais, oferecendo um excelente equilíbrio entre normalização e praticidade.

Violação da 3FN - Exemplo

ID_Funcionario	Nome_Departamento	ID_Departamento	Nome_Departamento	Localizacao_Depto
1	João Silva	10	TI	Prédio A
2	Maria Santos	10	TI	Prédio A
3	Pedro Costa	20	RH	Prédio B

Problema: Nome_Departamento e Localizacao_Depto dependem de ID_Departamento, não diretamente de ID_Funcionario. Há dependência transitiva: ID_Funcionario ³ ID_Departamento ³ Nome_Departamento.

Normalização em 3FN - Solução

Tabela: Departamento

Tabela: Funcionário

ID_Funcionario	Nome ID_Departamento	ID_Departamento Nome Localização
1 João Silva	10	10 TI Prédio A
2 Maria Santos	20	armazenadas uma única vez, eliminando redundância e dependência transitiva.
3 Pedro Costa	20	RH Prédio B
Informações do departamento		

Cada funcionário referencia seu departamento através da chave estrangeira ID_Departamento.

Benefícios: Atualizar o nome ou localização de um departamento requer alterar apenas um registro; não há risco de inconsistência; economia de espaço de armazenamento; estrutura mais flexível para consultas e manutenção.

Cláusula WHERE - Filtrando Dados

A cláusula WHERE é fundamental em SQL para filtrar registros em consultas, retornando apenas as linhas que satisfazem condições específicas. Ela permite extrair subconjuntos relevantes de dados de tabelas que podem conter milhões de registros. WHERE atua como um filtro lógico, avaliando cada linha da tabela contra a condição especificada e incluindo no resultado apenas aquelas onde a condição é verdadeira.

A sintaxe básica de WHERE envolve especificar uma ou mais condições usando operadores de comparação (=, !=, >, <, >=, <=) e operadores lógicos (AND, OR, NOT) para combinar múltiplas condições. As condições podem envolver comparações entre colunas e valores literais, entre colunas, ou expressões mais complexas envolvendo funções. O WHERE é aplicado antes de qualquer agrupamento ou ordenação, tornando-o eficiente para reduzir o volume de dados processados.

```
-- Sintaxe básica
SELECT coluna1, coluna2
FROM tabela
WHERE condição;
```

```
-- Exemplos práticos
SELECT Nome, Email, Idade
FROM Aluno
WHERE Idade >= 18;
```

```
SELECT Produto, Preco, Estoque
FROM Produtos
WHERE Categoria = 'Eletrônicos' AND Preco < 1000;
```

```
SELECT *
FROM Pedidos
WHERE Data_Pedido BETWEEN '2024-01-01' AND '2024-12-31';
```

```
SELECT Nome, Salario
FROM Funcionarios
WHERE Departamento = 'TI' OR Salario > 5000;
```

WHERE pode usar diversos operadores especiais como BETWEEN (para intervalos), IN (para listas de valores), LIKE (para padrões de texto), IS NULL (para valores nulos), e NOT para negação. Estes operadores tornam as consultas mais expressivas e legíveis.

Operadores na Cláusula WHERE

Operadores de Comparação

= (igual), != ou <>
(diferente) > (maior), <
(menor)

>= (maior ou igual),
<= (menor ou igual)

```
WHERE Idade >= 18
WHERE Status !=
'Inativo'
```

Operadores Lógicos

AND - todas condições
verdadeiras

OR - pelo menos uma
verdadeira

NOT - negação da condição

```
WHERE Cidade = 'SP'
```

```
AND Ativo = 1
```

```
WHERE Status = 'A' OR
Status = 'B'
```

```
WHERE NOT Categoria
= 'Descontinuado'
```

Operadores Especiais

BETWEEN - intervalo de

valores

IN - lista de valores
possíveis

LIKE - padrões de texto

IS NULL / IS NOT NULL -
valores nulos

```
WHERE Preco BETWEEN
100 AND 500
```

```
WHERE Estado IN ('SP',
'RJ', 'MG')
```

```
WHERE Email IS NOT
NULL
```

Cláusula LIKE - Buscas por Padrão

O operador LIKE é usado na cláusula WHERE para buscar padrões em campos de texto, permitindo consultas flexíveis quando não conhecemos o valor exato ou queremos encontrar valores

similares. LIKE utiliza caracteres curinga para representar partes variáveis do padrão: o símbolo percentual (%) representa zero ou mais caracteres, enquanto o underscore (_) representa exatamente um caractere.

```
WHERE Email LIKE '%@gmail.com';
```

Padrões com %

```
-- Nomes que começam com  
'João' SELECT Nome FROM Aluno  
WHERE Nome LIKE 'João%';
```

```
-- Nomes que terminam com  
'Silva' SELECT Nome FROM Aluno  
WHERE Nome LIKE '%Silva';
```

```
-- Nomes que contêm  
'Maria' SELECT Nome FROM  
Aluno  
WHERE Nome LIKE '%Maria%';
```

```
-- Emails do domínio gmail  
SELECT Email FROM Usuario
```

Padrões com _

```
-- Códigos com formato XXX-XXXX  
SELECT Codigo FROM Produto  
WHERE Codigo LIKE '___-____';
```

```
-- Nomes com 4 letras  
SELECT Nome FROM Cidade  
WHERE Nome LIKE '____';
```

```
-- CEP com formato XXXXX-XXX  
SELECT CEP FROM Endereco  
WHERE CEP LIKE '____-__';
```

Nota: LIKE é case-insensitive por padrão no MySQL, mas pode ser sensível a maiúsculas dependendo da collation da tabela.

Cláusula GROUP BY - Agrupando Dados

A cláusula GROUP BY agrupa linhas que possuem valores iguais em colunas especificadas, permitindo realizar cálculos agregados sobre cada grupo. É fundamental para análise de dados e geração de relatórios sumarizados. GROUP BY é sempre usado em conjunto com funções de agregação como COUNT(), SUM(), AVG(), MAX() e MIN(), que calculam valores sobre os grupos formados.

Quando usamos GROUP BY, o resultado da consulta retorna uma linha para cada grupo único de valores nas colunas de agrupamento. Todas as colunas no SELECT devem estar ou na cláusula GROUP BY ou dentro de uma função de agregação. Esta restrição garante que o resultado seja consistente e bem definido.

```
-- Sintaxe básica  
SELECT coluna_agrupamento, FUNCAO_AGREGACAO(coluna)  
FROM tabela  
GROUP BY coluna_agrupamento;
```

```
-- Contar alunos por curso  
SELECT Curso, COUNT(*) as Total_Alunos  
FROM Aluno
```

GROUP BY Curso;

-- Média de notas por disciplina

```
SELECT Disciplina, AVG(Nota) as Media
FROM Matricula
GROUP BY Disciplina;
```

-- Total de vendas por vendedor e mês

```
SELECT Vendedor, MONTH(Data_Venda) as Mes, SUM(Valor) as Total
FROM Vendas
GROUP BY Vendedor, MONTH(Data_Venda);
```

-- Produtos por categoria com estoque total

```
SELECT Categoria, COUNT(*) as Qtd_Produtos, SUM(Estoque) as
Estoque_Total FROM Produtos
GROUP BY Categoria;
```

Funções de Agregação

COUNT()

Conta o número de linhas ou valores não-nulos
Pedidos

GROUP BY Cliente_ID;

AVG()

Calcula a média
aritmética dos valores

```
SELECT Departamento,
COUNT(*) as
Total_Func
FROM Funcionario
GROUP BY
Departamento;
```

```
SELECT Disciplina,
AVG(Nota) as
Media_Turma FROM
Notas
GROUP BY Disciplina;
```

SUM()

Soma valores
numéricos de um
grupo

```
SELECT Cliente_ID,
SUM(Valor) as
Total_Gasto FROM
```

MAX()

Retorna o valor
máximo do grupo

```
SELECT Categoria,
MAX(Preco) as
Preco_Maximo FROM
```

Produtos
GROUP BY Categoria; Retorna o valor
mínimo do grupo

MIN()

Cláusula HAVING - Filtrando Grupos

A cláusula HAVING é usada para filtrar grupos criados pelo GROUP BY, funcionando como um WHERE para dados agregados. Enquanto WHERE filtra linhas individuais antes do agrupamento, HAVING filtra grupos após a agregação. Isto permite selecionar apenas grupos que satisfazem condições específicas baseadas em funções de agregação.

WHERE vs HAVING

WHERE:

- Filtra linhas individuais
- Aplicado antes do GROUP BY
- Não pode usar funções de agregação
- Mais eficiente (reduz dados processados)

HAVING:

- Filtra grupos agregados
- Aplicado após o GROUP BY
- Pode usar funções de agregação
- Usado para condições sobre resultados agregados

Exemplos Práticos

-- Departamentos com mais de 5

funcionários

```
SELECT Departamento, COUNT(*) as  
Total FROM Funcionario  
GROUP BY Departamento  
HAVING COUNT(*) > 5;
```

-- Produtos com média de preço acima de 100

```
SELECT Categoria, AVG(Preco) as  
Media FROM Produtos  
GROUP BY Categoria  
HAVING AVG(Preco) > 100;
```

-- Clientes que gastaram mais de 1000

```
SELECT Cliente_ID, SUM(Valor) as  
Total FROM Pedidos  
WHERE Data_Pedido >=  
'2024-01-01' GROUP BY Cliente_ID  
HAVING SUM(Valor) > 1000;
```

Note no último exemplo como WHERE e HAVING trabalham juntos: WHERE filtra pedidos de 2024, depois GROUP BY agrupa por cliente, e HAVING seleciona apenas clientes com total acima de 1000.

Cláusula ORDER BY - Ordenando Resultados

A cláusula ORDER BY ordena o resultado de uma consulta com base em uma ou mais colunas, em

ordem crescente (ASC) ou decrescente (DESC). Ordenar dados é essencial para apresentação legível de resultados e para facilitar análise de informações. ORDER BY é sempre a última cláusula executada em uma consulta SELECT, após WHERE, GROUP BY e HAVING.

-- Sintaxe básica

```
SELECT colunas  
FROM tabela  
ORDER BY coluna1 [ASC | DESC], coluna2 [ASC | DESC];
```

-- Ordenar alunos por nome (crescente é padrão)

```
SELECT Nome, Idade  
FROM Aluno  
ORDER BY Nome;
```

-- Ordenar produtos por preço decrescente

```
SELECT Nome, Preco  
FROM Produtos  
ORDER BY Preco DESC;
```

-- Ordenar por múltiplas colunas

```
SELECT Departamento, Nome, Salario  
FROM Funcionarios  
ORDER BY Departamento ASC, Salario DESC;
```

-- Ordenar usando posição da coluna

```
SELECT Nome, Email, Cidade  
FROM Usuarios  
ORDER BY 3, 1; -- Ordena por Cidade (col 3), depois Nome (col 1)
```

-- Ordenar com expressões

```
SELECT Nome, Quantidade * Preco as Total  
FROM Itens_Pedido  
ORDER BY Quantidade * Preco DESC;
```

ORDER BY - Técnicas Avançadas

Ordenação com NULL

Valores NULL aparecem primeiro em ordem ascendente e por último em ordem descendente por padrão. Use NULLS FIRST ou NULLS LAST para controlar.

```
SELECT Nome, Telefone  
FROM Contatos
```

ORDER BY Telefone DESC NULLS LAST;

Ordenação Customizada com CASE

Use expressões CASE para criar ordenações personalizadas baseadas em lógica condicional.

```
SELECT Nome, Status
FROM Pedidos
ORDER BY CASE Status
  WHEN 'Urgente' THEN 1
  WHEN 'Prioritário' THEN 2
  WHEN 'Normal' THEN 3
  ELSE 4
END;
```

Ordenação com Agregação

Combine ORDER BY com GROUP BY para ordenar resultados agregados.

```
SELECT Categoria, COUNT(*) as Total
FROM Produtos
GROUP BY Categoria
ORDER BY Total DESC;
```

Limitando Resultados

SELECT de Múltiplas Tabelas - Introdução

Consultas que envolvem múltiplas tabelas são fundamentais em bancos de dados relacionais, permitindo combinar informações distribuídas em diferentes entidades. Existem duas abordagens principais: a sintaxe tradicional usando a cláusula WHERE para especificar condições de junção, e a sintaxe moderna usando JOIN explícito. Ambas atingem o mesmo resultado, mas JOIN é preferida por ser mais legível e menos propensa a erros.

Quando selecionamos de múltiplas tabelas, o banco de dados primeiro cria um produto cartesiano (todas as combinações possíveis de linhas entre as tabelas) e depois aplica os filtros especificados. Por isso é crucial especificar corretamente as condições de junção para evitar resultados incorretos ou excessivos. A condição de junção geralmente compara chaves estrangeiras com chaves primárias, estabelecendo o relacionamento lógico entre as tabelas.

Sintaxe Tradicional (WHERE)

-- Selecionar alunos e suas matrículas

```
SELECT A.Nome, M.Disciplina, M.Nota
FROM Aluno A, Matricula M
WHERE A.ID_Aluno = M.ID_Aluno;
```

-- Três tabelas: Pedido, Cliente e Produto

```
SELECT C.Nome, P.Data_Pedido, Pr.Nome_Produto
FROM Cliente C, Pedido P, Item_Pedido I, Produto Pr
WHERE C.ID_Cliente = P.ID_Cliente
AND P.ID_Pedido = I.ID_Pedido
AND I.ID_Produto = Pr.ID_Produto;
```

Note o uso de aliases (A, M, C, P, etc.) para abreviar nomes de tabelas e tornar a consulta mais legível. Aliases são especialmente úteis quando trabalhamos com múltiplas tabelas.

Tipos de JOIN

JOIN é a operação que combina linhas de duas ou mais tabelas baseadas em uma coluna relacionada entre elas. Existem diferentes tipos de JOIN, cada um com comportamento específico quanto a quais linhas são incluídas no resultado.

INNER JOIN

Retorna apenas linhas que têm correspondência em ambas as tabelas.

É o tipo mais comum e é o padrão quando usamos apenas JOIN.

```
SELECT A.Nome, M.Nota
FROM Aluno A
INNER JOIN Matricula M
ON A.ID_Aluno = M.ID_Aluno;
```

RIGHT JOIN

Retorna todas as linhas da tabela à direita e as correspondentes da esquerda. Oposto do LEFT JOIN.

```
SELECT A.Nome, M.Nota
FROM Aluno A
RIGHT JOIN Matricula M
ON A.ID_Aluno = M.ID_Aluno;
```

LEFT JOIN

Retorna todas as linhas da tabela à esquerda e as correspondentes da direita. Se não houver correspondência, retorna NULL para colunas da direita.

mas pode ser simulado com UNION.

```
SELECT A.Nome, M.Nota
FROM Aluno A
LEFT JOIN Matricula M
ON A.ID_Aluno = M.ID_Aluno;
```

FULL OUTER JOIN

Retorna todas as linhas quando há correspondência em uma das tabelas.
Não suportado nativamente no MySQL,

```
SELECT A.Nome, M.Nota
FROM Aluno A
LEFT JOIN Matricula M
ON A.ID_Aluno = M.ID_Aluno
UNION
SELECT A.Nome, M.Nota
FROM Aluno A
RIGHT JOIN Matricula M
ON A.ID_Aluno = M.ID_Aluno;
```

INNER JOIN - Exemplos Práticos

INNER JOIN é usado quando queremos apenas registros que têm correspondência em ambas as tabelas. É ideal para consultas onde a relação entre tabelas é obrigatória.

JOIN de Três Tabelas

JOIN de Duas Tabelas

```
-- Listar pedidos com nome do
cliente SELECT
P.ID_Pedido,
P.Data_Pedido,
C.Nome as Cliente,
P.Valor_Total
FROM Pedido P
INNER JOIN Cliente C
ON P.ID_Cliente = C.ID_Cliente
ORDER BY P.Data_Pedido DESC;
```

Esta consulta retorna apenas pedidos que têm um cliente associado.

```
-- Listar produtos vendidos com
detalhes SELECT
P.Nome_Produto,
C.Categoria,
IP.Quantidade,
IP.Preco_Unitario,
(IP.Quantidade * IP.Preco_Unitario) as
Subtotal
FROM Item_Pedido IP
INNER JOIN Produto P
ON IP.ID_Produto = P.ID_Produto
INNER JOIN Categoria C
ON P.ID_Categoria = C.ID_Categoria
WHERE IP.ID_Pedido = 100;
```

Cada INNER JOIN adiciona uma nova tabela à consulta, expandindo as informações disponíveis. A ordem dos JOINS geralmente não afeta o resultado final, apenas a legibilidade.

LEFT JOIN - Incluindo Todos os Registros

LEFT JOIN é usado quando queremos manter todos os registros da tabela à esquerda, mesmo aqueles sem correspondência na tabela à direita. É útil para identificar registros órfãos ou gerar

relatórios completos.

-- Listar todos os alunos, incluindo os sem matrícula

```
SELECT
  A.ID_Aluno,
  A.Nome,
  A.Email,
  COUNT(M.ID_Matricula) as Total_Matriculas
FROM Aluno A
LEFT JOIN Matricula M ON A.ID_Aluno = M.ID_Aluno
GROUP BY A.ID_Aluno, A.Nome, A.Email
ORDER BY Total_Matriculas;
```

-- Encontrar clientes sem pedidos

```
SELECT
  C.ID_Cliente,
  C.Nome,
  C.Email
FROM Cliente C
LEFT JOIN Pedido P ON C.ID_Cliente = P.ID_Cliente
WHERE P.ID_Pedido IS NULL;
```

-- Listar produtos e suas vendas (incluindo não vendidos)

```
SELECT
  P.Nome_Produto,
  P.Estoque,
  COALESCE(SUM(IP.Quantidade), 0) as Total_Vendido
FROM Produto P
LEFT JOIN Item_Pedido IP ON P.ID_Produto = IP.ID_Produto
GROUP BY P.ID_Produto, P.Nome_Produto, P.Estoque;
```

Note o uso de IS NULL para filtrar registros sem correspondência e COALESCE para substituir NULL por zero em cálculos.

Tipos de Dados no SQL

Tipos de dados definem que tipo de valores podem ser armazenados em cada coluna de uma tabela. Escolher o tipo de dado correto é crucial para otimizar espaço de armazenamento, garantir integridade dos dados e melhorar performance das consultas. MySQL oferece uma ampla variedade de tipos de dados agrupados em categorias: numéricos, texto, data/hora, e especiais.

A escolha inadequada do tipo de dado pode levar a desperdício de espaço (usar INT para um valor que nunca passa de 100), perda de precisão (usar FLOAT quando se precisa de DECIMAL para valores monetários), ou limitações funcionais (usar VARCHAR quando ENUM seria mais apropriado). Além disso, o tipo de dado afeta quais operações e funções podem ser aplicadas à coluna.

Tipos de Dados Numéricos

Tipo Tamanho Intervalo Uso Recomendado

TINYINT 1 byte -128 a 127 (ou 0 a 255 unsigned) Valores pequenos, flags booleanas

SMALLINT 2 bytes -32.768 a 32.767 Contadores, quantidades
pequenas

INT 4 bytes -2.147.483.648 a 2.147.483.647 IDs, quantidades gerais

BIGINT 8 bytes Números muito grandes IDs em tabelas muito grandes

DECIMAL(M,D) Variável Precisão exata até M dígitos Valores monetários, financeiros

FLOAT 4 bytes Números com precisão simples Valores científicos aproximados

DOUBLE 8 bytes Números com precisão dupla Cálculos complexos, coordenadas

-- Exemplos de uso

```
CREATE TABLE Produto (  
  ID_Produto INT AUTO_INCREMENT PRIMARY KEY,  
  Nome VARCHAR(100),  
  Preco DECIMAL(10,2), -- Até 999999999.99  
  Estoque SMALLINT UNSIGNED,  
  Peso FLOAT,  
  Ativo TINYINT(1) -- 0 ou 1 como booleano  
);
```

Tipos de Dados de Texto

CHAR(n)

Tamanho fixo: sempre usa n bytes

Preenche com espaços se valor

menor **Uso:** Códigos fixos (CPF, CEP, siglas)

CEP CHAR(9) -- '01310-100'

UF CHAR(2) -- 'SP'

TEXT

Textos longos: até 65.535 caracteres

Não permite DEFAULT, mais lento que VARCHAR

Uso: Comentários, descrições longas

Descricao TEXT

Observacoes TEXT

VARCHAR(n)

Tamanho variável: usa apenas espaço necessário + 1-2 bytes

Máximo de 65.535 caracteres

Uso: Nomes, descrições, emails

Nome VARCHAR(100)

Email VARCHAR(255)

Textos muito longos: até

4GB Uso intensivo de

memória

Uso: Artigos completos, documentos

Conteúdo LONGTEXT

Documento LONGTEXT

LONGTEXT

Dica: Prefira VARCHAR para a maioria dos casos. Use TEXT apenas quando necessário, pois tem overhead de performance.

Tipos de Dados de Data e Hora

'2038- 01-19 03:14:07'

Tamanho: 4 bytes

Atualizado automaticamente

Uso: Controle de

criação/modificação **YEAR** - Apenas ano

Range: 1901 a 2155

Tamanho: 1 byte

Uso: Ano de fabricação, ano letivo

```
CREATE TABLE Evento (  
  ID INT PRIMARY KEY,  
  Nome VARCHAR(100),  
  Data_Evento DATE,  
  Hora_Inicio TIME,  
  Criado_Em TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP,  
  Modificado_Em TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP  
  ON UPDATE CURRENT_TIMESTAMP  
);
```

Tipos Disponíveis

DATE - Apenas data (YYYY-MM-DD)

Range: '1000-01-01' a

'9999-12-31' Tamanho: 3 bytes

Uso: Datas de nascimento, datas de vencimento

TIME - Apenas hora (HH:MM:SS)

Range: '-838:59:59' a '838:59:59'

Tamanho: 3 bytes

Uso: Horários de funcionamento,

durações **DATETIME** - Data e hora

Range: '1000-01-01 00:00:00' a

'9999-12- 31 23:59:59'

Tamanho: 8 bytes

Uso: Timestamps gerais, datas de eventos

TIMESTAMP - Data e hora com timezone

Range: '1970-01-01 00:00:01' UTC a

VIEW - Visões em Banco de Dados

Uma VIEW (visão) é uma tabela virtual baseada no resultado de uma consulta SQL. Não armazena dados fisicamente, mas apresenta dados de uma ou mais tabelas subjacentes de forma organizada e simplificada. VIEWS são extremamente úteis para encapsular consultas complexas, aumentar segurança restringindo acesso a colunas específicas, e simplificar a interação de usuários com o banco de dados.

VIEWS funcionam como "janelas" para os dados reais. Quando você consulta uma VIEW, o MySQL executa a consulta definida na VIEW e retorna os resultados como se fossem uma tabela real. Alterações nos dados das tabelas base são imediatamente refletidas na VIEW. Além disso, algumas VIEWS podem ser atualizáveis, permitindo INSERT, UPDATE e DELETE através delas, desde que satisfaçam certas condições.

-- Criar uma VIEW simples

```
CREATE VIEW View_Alunos_Ativos AS
SELECT ID_Aluno, Nome, Email, Curso
FROM Aluno
WHERE Status = 'Ativo';
```

-- Usar a VIEW como uma tabela

```
SELECT * FROM View_Alunos_Ativos
WHERE Curso = 'Ciência da Computação';
```

-- VIEW com JOIN de múltiplas tabelas

```
CREATE VIEW View_Pedidos_Completos AS
SELECT
P.ID_Pedido,
P.Data_Pedido,
C.Nome as Cliente,
C.Email,
P.Valor_Total,
P.Status
FROM Pedido P
INNER JOIN Cliente C ON P.ID_Cliente = C.ID_Cliente;
```

-- Modificar uma VIEW existente

```
CREATE OR REPLACE VIEW View_Alunos_Ativos AS
SELECT ID_Aluno, Nome, Email, Curso, Data_Matricula
FROM Aluno
WHERE Status = 'Ativo' AND Data_Matricula >= '2024-01-01';
```

Vantagens e Uso de VIEWS



Segurança

Restringir acesso a colunas sensíveis, expondo apenas dados necessários aos usuários. Usuários podem acessar a VIEW sem ver a estrutura completa das tabelas.

```
CREATE VIEW View_Func_Publico AS
SELECT Nome, Departamento,
Email FROM Funcionario;
-- Oculta Salario, CPF, etc.
```

Encapsular consultas complexas com múltiplos JOINS e condições em uma interface simples. Usuários consultam a VIEW sem conhecer a complexidade subjacente.

```
CREATE VIEW View_Vendas_Mes AS
SELECT DATE_FORMAT(Data, '%Y-%m') as
Mes,
SUM(Valor) as Total
FROM Vendas
GROUP BY DATE_FORMAT(Data,
'%Y- %m');
```

Independência Lógica

Isolar aplicações de mudanças na estrutura do banco. Se tabelas mudarem, apenas a VIEW precisa ser ajustada, não as aplicações que a usam.

Simplificação

Reutilização

Definir consultas frequentes uma vez e reutilizá-las em múltiplas aplicações e relatórios, garantindo consistência e reduzindo duplicação de código.

VIEWS são ferramentas poderosas para organização e governança de dados. Use-as estrategicamente para melhorar manutenibilidade e segurança do sistema.

TRIGGER - Gatilhos Automáticos

Um TRIGGER (gatilho) é um procedimento armazenado que é executado automaticamente em resposta a eventos específicos em uma tabela, como INSERT, UPDATE ou DELETE. TRIGGERS permitem automatizar ações, manter integridade de dados complexa, auditar alterações e implementar regras de negócio diretamente no banco de dados. São fundamentais para garantir consistência sem depender da aplicação.

TRIGGERS podem ser definidos para executar BEFORE (antes) ou AFTER (depois) do evento que os aciona. Um TRIGGER BEFORE pode modificar os dados antes de serem inseridos ou atualizados, enquanto um TRIGGER AFTER pode realizar ações baseadas nos dados já confirmados. Cada TRIGGER é associado a uma tabela específica e um evento específico (INSERT, UPDATE ou DELETE).

-- TRIGGER para auditoria: registrar todas as alterações

```

CREATE TRIGGER Audit_Produto_Update
AFTER UPDATE ON Produto
FOR EACH ROW
BEGIN
INSERT INTO Log_Alteracoes (
Tabela,
ID_Registro,
Usuario,
Data_Hora,
Valor_Anterior,
Valor_Novo
) VALUES (
'Produto',
NEW.ID_Produto,
USER(),
NOW(),
OLD.Preco,
NEW.Preco
);
END;

```

```

-- TRIGGER para validação antes de inserir
CREATE TRIGGER Valida_Estoque_Antes_Venda
BEFORE INSERT ON Item_Pedido
FOR EACH ROW
BEGIN
DECLARE estoque_atual INT;

SELECT Estoque INTO estoque_atual
FROM Produto
WHERE ID_Produto = NEW.ID_Produto;

```

Exemplos Práticos de TRIGGERS

Atualização Automática de Totais

```

-- Atualizar total do pedido
automaticamente
CREATE TRIGGER
Atualiza_Total_Pedido AFTER INSERT
ON Item_Pedido
FOR EACH ROW
BEGIN

```

```

UPDATE Pedido
SET Valor_Total = (
SELECT SUM(Quantidade *
Preco_Unitario)
FROM Item_Pedido
WHERE ID_Pedido = NEW.ID_Pedido )
WHERE ID_Pedido = NEW.ID_Pedido;
END;

```

```

-- Atualizar estoque após venda
CREATE TRIGGER
Atualiza_Estoque_Venda AFTER INSERT

```

```

ON Item_Pedido
FOR EACH ROW
BEGIN
UPDATE Produto
SET Estoque = Estoque - NEW.Quantidade
WHERE ID_Produto = NEW.ID_Produto;
END;

```

Controle de Timestamps

```

-- Definir data de criação
automaticamente CREATE TRIGGER
Set_Data_Criacao BEFORE INSERT ON
Cliente
FOR EACH ROW
BEGIN
SET NEW.Data_Cadastro = NOW(); SET
NEW.Data_Modificacao = NOW(); END;

```

```

-- Atualizar data de modificação
CREATE TRIGGER
Update_Data_Modificacao BEFORE
UPDATE ON Cliente
FOR EACH ROW
BEGIN
SET NEW.Data_Modificacao = NOW();
END;

```

```

-- Remover TRIGGERS
DROP TRIGGER IF EXISTS
Atualiza_Total_Pedido;

-- Visualizar TRIGGERS existentes
SHOW TRIGGERS FROM nome_database;

```

Cuidado: TRIGGERS podem impactar performance se executarem operações complexas. Use-os criteriosamente e monitore o desempenho.

STORED PROCEDURE - Procedimentos Armazenados

Stored Procedures (procedimentos armazenados) são conjuntos de comandos SQL salvos no banco de dados que podem ser executados como uma unidade. Funcionam como funções ou métodos em programação, podendo receber parâmetros, executar lógica complexa, retornar resultados e serem reutilizados. São fundamentais para encapsular lógica de negócio no banco de dados, melhorar performance e centralizar operações comuns.

Stored Procedures oferecem múltiplas vantagens: redução de tráfego de rede (executam no servidor sem enviar múltiplos comandos), reutilização de código, melhoria de segurança (usuários podem executar procedures sem acesso direto às tabelas), e melhor performance (código compilado e otimizado pelo SGBD). Além disso, facilitam manutenção centralizada de lógica de negócio.

```

-- Procedure simples sem parâmetros
DELIMITER //
CREATE PROCEDURE Listar_Clientes_Ativos()
BEGIN

```

```

SELECT ID_Cliente, Nome, Email, Cidade
FROM Cliente
WHERE Status = 'Ativo'
ORDER BY Nome;
END //
DELIMITER ;

-- Executar a procedure
CALL Listar_Clientes_Ativos();

-- Procedure com parâmetros de entrada
DELIMITER //
CREATE PROCEDURE Buscar_Produtos_Por_Categoria(
  IN categoria_nome VARCHAR(50),
  IN preco_maximo DECIMAL(10,2)
)
BEGIN
  SELECT P.Nome_Produto, P.Preco, P.Estoque
  FROM Produto P
  INNER JOIN Categoria C ON P.ID_Categoria = C.ID_Categoria
  WHERE C.Nome = categoria_nome AND P.Preco <= preco_maximo
  ORDER BY P.Preco;
END //
DELIMITER ;

```

Stored Procedures com Parâmetros OUT

```

-- Procedure com parâmetros OUT (retornam valores)
DELIMITER //
CREATE PROCEDURE Calcular_Estatisticas_Vendas(
  IN mes INT,
  IN ano INT,
  OUT total_vendas DECIMAL(10,2),
  OUT qtd_pedidos INT,
  OUT ticket_medio DECIMAL(10,2)
)
BEGIN
  SELECT
    SUM(Valor_Total),
    COUNT(*),
    AVG(Valor_Total)
  INTO total_vendas, qtd_pedidos, ticket_medio

```

```

FROM Pedido
WHERE MONTH(Data_Pedido) = mes
AND YEAR(Data_Pedido) = ano;
END //
DELIMITER ;

-- Executar e obter valores de retorno
CALL Calcular_Estatisticas_Vendas(5, 2024, @total, @qtd, @media);
SELECT @total as Total_Vendas, @qtd as Quantidade_Pedidos, @media as Ticket_Medio;

-- Procedure com lógica condicional
DELIMITER //
CREATE PROCEDURE Aplicar_Desconto_Cliente(
    IN cliente_id INT,
    IN percentual_desconto DECIMAL(5,2)
)
BEGIN
    DECLARE total_compras DECIMAL(10,2);

    -- Calcular total de compras do cliente
    SELECT SUM(Valor_Total) INTO total_compras
    FROM Pedido
    WHERE ID_Cliente = cliente_id AND Status = 'Finalizado';

    -- Aplicar desconto apenas se cliente for premium (>5000 em compras)
    IF total_compras > 5000 THEN

```

RAID em Bancos de Dados

RAID (Redundant Array of Independent Disks) é uma tecnologia que combina múltiplos discos físicos em uma unidade lógica para melhorar performance, confiabilidade ou ambos. Embora RAID seja uma tecnologia de infraestrutura e não de banco de dados em si, é crucial para sistemas que exigem alta disponibilidade e desempenho. Diferentes níveis de RAID oferecem diferentes trade offs entre velocidade, redundância e capacidade de armazenamento.

Para bancos de dados críticos, RAID proporciona proteção contra falhas de disco, melhora velocidades de leitura/escrita através de paralelização, e garante continuidade de operação mesmo quando um disco falha. A escolha do nível de RAID apropriado depende dos requisitos específicos de cada sistema: algumas aplicações priorizam velocidade (RAID 0), outras priorizam segurança (RAID 1), e muitas buscam um equilíbrio entre ambos (RAID 5, RAID 10).

Níveis de RAID Principais

	Método: Dados	Vantagens: Máxima	Desvantagens: Sem
RAID 0 - Striping	divididos entre discos	velocidade, uso total da capacidade	redundância, falha de um disco perde tudo

Uso: Ambientes de desenvolvimento, cache temporário

RAID 1 - Mirroring

Método: Dados duplicados em discos espelhados

Vantagens: Alta confiabilidade, leitura rápida

Desvantagens: Usa 50% da capacidade total

Uso: Sistemas críticos, logs de transações

RAID 5 - Parity

Método: Striping com paridade distribuída

Vantagens: Equilíbrio

custo/performance/s e gurança

Desvantagens:

Escrita mais lenta, rebuilds demorados

Uso: Servidores de aplicação, bancos OLTP

RAID 10 - Mirror+Stripe

Método: Combina RAID 1 e RAID 0

Vantagens: Alta performance e redundância

Desvantagens: Caro, usa 50% da capacidade

Uso: Bancos de dados de alta demanda

MySQL Workbench - Ferramenta Visual

MySQL Workbench é uma ferramenta visual unificada para arquitetos de banco de dados, desenvolvedores e DBAs. Oferece modelagem de dados, desenvolvimento SQL, administração de servidor e migração de dados em um ambiente integrado. É a ferramenta oficial da Oracle para trabalhar com MySQL, disponível gratuitamente para Windows, Linux e macOS.

Modelagem de Dados

Crie diagramas ER visualmente, defina tabelas, relacionamentos, índices e constraints usando interface gráfica intuitiva. Gere automaticamente scripts SQL a partir dos modelos.

Administração

Gerencie usuários, permissões, backups, importação/exportação de dados, monitoramento de performance e otimização de consultas através de interface visual.

Editor SQL

Escreva e execute consultas SQL com recursos avançados: autocompletar, syntax highlighting, formatação de código, histórico de consultas e resultados exportáveis.

Migração de Dados

Migre bancos de dados de diferentes SGBDs (Microsoft SQL Server, PostgreSQL, etc.) para MySQL com assistente visual que converte esquemas e dados automaticamente.

Conclusão e Próximos Passos

Parabéns por completar este guia abrangente sobre bancos de dados relacionais e MySQL Workbench! Você agora possui conhecimento sólido dos fundamentos: desde conceitos básicos como entidades e relacionamentos, passando por normalização e modelagem, até técnicas avançadas como JOINS, stored procedures, triggers e views.



Pratique Constantemente

A melhor forma de consolidar conhecimento é através da prática. Crie seus próprios bancos de dados, implemente os conceitos aprendidos e experimente com diferentes cenários.

Desenvolva Projetos Reais

Aplique seus conhecimentos em projetos práticos: sistema de biblioteca, e-commerce, controle de estoque. Projetos reais revelam desafios e soluções que apenas teoria não

Aprofunde-se
ensina.

Gradualmente

Explore tópicos avançados: otimização de consultas, índices, transações, locks, particionamento, replicação. O aprendizado em banco de dados é uma jornada contínua.

Lembre-se que as normas ABNT devem ser seguidas na formatação de seu trabalho acadêmico: utilize fonte Arial ou Times New Roman 12, espaçamento 1,5, margens de 3cm (superior e esquerda) e 2cm (inferior e direita), e siga as diretrizes para citações e referências. Consulte a NBR 14724 para trabalhos acadêmicos e NBR 6023 para referências bibliográficas.

Este material serve como base sólida para seu TCC e sua carreira profissional com bancos de dados. Continue estudando, praticando e explorando. O domínio de bancos de dados relacionais é uma habilidade valiosa e permanente no mercado de tecnologia. Sucesso em sua jornada!